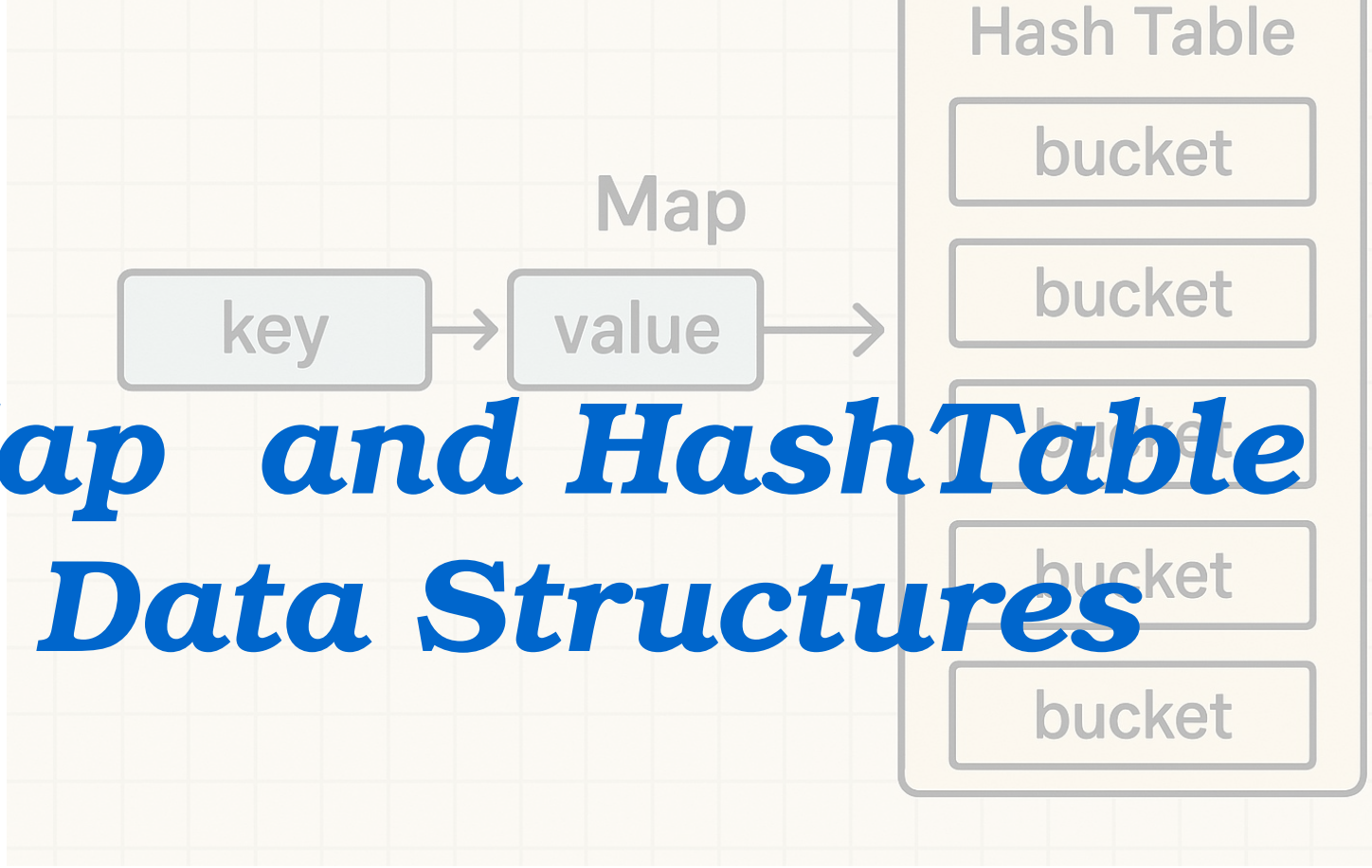


Map and HashTable Data Structures

Map: Definition

Maps (also known as Dictionaries) are data structures stores a collection of **key-value** pairs. Each key is **unique** and allows for **quick access to values.** A real life example of a map could be storing the grades for students in a class (student name is key, grade is value).

By default, it stores data in ascending order of the keys, but this can be changes as per requirement

In C++ and many other languages, a **map** is implemented in the Standard Library as an **associative container.**

Map is dictionary

Map is compared to a **dictionary where a Word** = key and its definition=value
The term **“dictionary”** is simply another name for the same abstract data structure that C++ calls a **“map.”** Both provide an **associative array** (or key-value store) interface:

- **Map** (C++, Java, etc.)
  Emphasizes the idea of “mapping” each key to a value.

- **Dictionary** (Python, C#, VB, etc.)
  Conjures the analogy of a real-world dictionary: you look up a word (the key) to find its definition (the value).

Roles in Data Storage

1. **Efficient Retrieval**:
   - You don’t search through all the data.
   - You use the **key to access the value directly** (like a phonebook).

2. **Organized Data Representation**:
   - Keys can be sorted (e.g., in C++ `map`) or hashed (e.g., `unordered_map`).
   - Data is easy to group, classify, and retrieve.

3. **Indexing**:
   - Think of keys as unique indexes for fast access.
   - Used in databases, caches, configuration settings, etc.

Examples of usage

Use case	Key	Value
User authentication	Username	Password hash
Phone directory	Name	Phone number
Word frequency counter	Word	Number of occurrences
Configuration settings	Setting name	Setting value
Web cache	URL	Cached page content
Banking app	Acount	Balance
Airline Booking	TicketNumber	Passenger Info

Map implementation in C++

C++ offers two primary map implementations:

1. **std::map:** This is the standard ordered map, providing the features mentioned above. It uses a **red black tree** internally for efficient Sorting and lookup operations.
2. **std::unordered_map:** This is an unordered map that uses a hash table for faster lookup operations. However, elements are not stored in any specific order.

Choosing between **std::map** and **std::unordered_map** depends on your specific needs. If you require ordered elements and are willing to sacrifice some lookup speed, **std::map** is a good choice. If lookup performance is critical

and order is not important, **std::unordered_map** is a better

Map in Java

Map Type	Key Features	Best Use Case
HashMap	- Fast lookups via hash table- No order guarantee	General-purpose map with fast access
TreeMap	- Keys sorted (natural or custom order)- Slower than HashMap	Sorted data access and range queries
LinkedHashMap	- Maintains insertion order- Predictable iteration	Order-sensitive applications
ConcurrentHashMap	- Thread-safe- Supports concurrent access without locking the whole map	High-performance multi-threaded environments
EnumMap	- Optimized for enum keys- Very efficient and compact	Maps with fixed set of enum keys

Map Operations

Operation	Example C++	Description
Insert	map[key] = value;	Add a new key-value pair to the map.
Retrieve	map.at(key) or map[key]	Get the value associated with a specific key.
Update	map[key] = newValue;	Change the value associated with an existing key.
Delete	map.erase(key);	Remove a key-value pair using the key.
Lookup	map.count(key) or map.find(key) != map.end()	Check if a key exists using count() or a default check.

Iteration	for (auto& pair : map) {
	cout << pair.first << ": " << pair.second << endl;
	}
		Traverse all key-value pairs using a loop or iterator.

Sorting		Sort by keys or values depending on map type (e.g., TreeMap).

Example1:C++

```
#include <iostream>
#include <map>
Using namespace std;
int main()
{
    // Creating a map
    map<string, int> myStock;

    // Inserting a new key-value pair
    myStock["apple"] = 100;
    myStock["banana"] = 200;
    myStock["cherry"] = 300;

    // Retrieving the value associated with a key
    int qty = myStock["banana"];
cout << "Value for key 'banana': " << qty << endl;
    // Updating the value associated with a key
    myStock["banana"] = 250;
    qty= myStock["banana"];
    cout << "Updated value for key 'banana': " << qty
            << endl;

    // Removing a key-value pair
    myStock.erase("cherry");

    // Iterating over the key-value pairs in the map
    cout << "Key-value pairs in the map:" << endl;
    for (const auto& pair : myStock) {
       cout << pair.first << ": " << pair.second
               << endl;
    }

    return 0;
}
```

Example 2:Java

```java
public class MapProgram {
    public static void main(String[]
args)
    {
        // Creating a map
        Map<String, Integer> myStock =
new HashMap<>();
        // Inserting a new key-value
pair
        myStock.put("apple", 100);
        myStock.put("banana", 200);
        myStock.put("cherry", 300);
       // Retrieving the value with a key
        int value =
myStock.get("banana");
        System.out.println("Value for key
'banana': "
                           + value);
        // Updating the value associated with a key
        myStock.put("banana", 250);
        value = myStock.get("banana");
        System.out.println(
            "Updated value for key 'banana': " +
value);
        // Removing a key-value pair
        myStock.remove("cherry");
        // Iterating over the key-value pairs in
the map
        System.out.println("Key-value pairs in the
map:");
        for (Map.Entry<String, Integer> pair :
            myStock.entrySet()) {
            System.out.println(pair.getKey() + ":
"
                               + pair.getValue());
        }
    }
}
```

Example 2:Java

```java
public class MapProgram {
    public static void main(String[]
args)
    {
        // Creating a map
        Map<String, Integer> myStock =
new HashMap<>();
        // Inserting a new key-value
pair
        myStock.put("apple", 100);
        myStock.put("banana", 200);
        myStock.put("cherry", 300);
       // Retrieving the value with a key
        int value =
myStock.get("banana");
        System.out.println("Value for key
'banana': "
                           + value);
        // Updating the value associated with a key
        myStock.put("banana", 250);
        value = myStock.get("banana");
        System.out.println(
            "Updated value for key 'banana': " +
value);
        // Removing a key-value pair
        myStock.remove("cherry");
        // Iterating over the key-value pairs in
the map
        System.out.println("Key-value pairs in the
map:");
        for (Map.Entry<String, Integer> pair :
            myStock.entrySet()) {
            System.out.println(pair.getKey() + ":
"
                               + pair.getValue());
        }
    }
}
```

Example 2: Java

```
// Iterating over the key-value pairs in the map
        System.out.println("Key-value pairs in the map:");
        for (Map.Entry<String, Integer> pair :
            myStock.entrySet()) {
             System.out.println(pair.getKey() + ": "
                                + pair.getValue());
        }
        //or
        // Iterate using keySet of a map
        System.out.println("Key-value pairs in the map:");
        for (String key:myStock.keySet()) {
             System.out.println(key + ": "
                                + myStock.get(key));
        }
```

Unordered Map

In C++, **unordered_map** is an unordered associative container that stores data in the form of unique key-value pairs. But unlike map, unordered map stores its elements using hashing. This provides average constant-time complexity **O(1)** for search, insert, and delete operations but the elements are not sorted in any particular order.

Examples: C++

```cpp
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {

    // Creating an unordered_map with integer
    // keys and string values
    unordered_map<int, string> um =
    {{6, "Rwanda"}, {10, "Coding"}, {3, "Academy"},{5, "C++"},{1, "Hello"}};

    for (auto i : um)
        cout << i.first << ": " << i.second
        << endl;
    return 0;
}
```

```
3: Academy
2: Coding
1: Rwanda
```

Custom Map using BST

```
template <typename K, typename V>

struct Node {

    K key;

    V value;

    Node* left;

    Node* right;

    Node(K k, V v) : key(k), value(v),
left(nullptr), right(nullptr) {}

};
```

```
#include <iostream>
using namespace std;

template <typename K, typename V>
class MyMap {
private:
    Node<K, V>* root = nullptr;

    Node<K, V>* insert(Node<K, V>* node, K key, V
value) {
        if (!node) return new Node<K, V>(key, value);

        if (key < node->key)
            node->left = insert(node->left, key, value);
        else if (key > node->key)
            node->right = insert(node->right, key, value);
        else
            node->value = value;  // update if key exists

        return node;
    }
```

Custom Map using BST

```
bool get(Node<K, V>* node, K key, V& out) const {
    if (!node) return false;

    if (key == node->key) {
        out = node->value;
        return true;
    } else if (key < node->key) {
        return get(node->left, key, out);
    } else {
        return get(node->right, key, out);
    }
}
void inorder(Node<K, V>* node) const {
    if (!node) return;
    inorder(node->left);
    cout << node->key << ": " << node->value << endl;
    inorder(node->right);
}
```

```
    void destroy(Node<K, V>* node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }
public:
    ~MyMap() {
        destroy(root);
    }
    void insert(K key, V value) {
        root = insert(root, key, value);
    }
    bool get(K key, V& out) const {
        return get(root, key, out);
    }
    void display() const {
        inorder(root);
    }
};
```

Main

```
int main() {
    MyMap<string, int> map;

    map.insert("Ben", 17);
    map.insert("Best", 15);
    map.insert("Babou", 25);
    map.display();  // Should print in sorted key order

    int val;
    if (map.get("Best", val)) {
        cout << "Best is " << val << " years old.\n";
    } else {
        cout << "Best not found.\n";
    }

    return 0;
}
```

Custom Map WIthout Generic

```
#include <iostream>
#include <string>

using namespace std;

// Node of the BST
struct Node {
    string key;
    int value;
    Node* left;
    Node* right;

    Node(string k, int v) : key(k), value(v)
left(NULL), right(NULL) {}
};
```

```
// Map class
class StringIntMap {
private:
    Node* root = NULLr;

    // Insert helper
    Node* insert(Node* node, const string& key, int
value) {
        if (!node) return new Node(key, value);

        if (key < node->key)
            node->left = insert(node->left, key, value);
        else if (key > node->key)
            node->right = insert(node->right, key, value);
        else
            node->value = value;  // update if key exists

    return node;
}
```

WIthout Generic

```
    // Get helper
    bool get(Node* node, const string& key, int& ou
{
        if (!node) return false;

        if (key == node->key) {
            out = node->value;
            return true;
        } else if (key < node->key) {
            return get(node->left, key, out);
        } else {
            return get(node->right, key, out);
        }
    }

    // In-order traversal (sorted display)
    void inorder(Node* node) const {
        if (!node) return;
        inorder(node->left);
        cout << node->key << ": " << node->value <<
        inorder(node->right);
    }
```

```
// Cleanup
    void destroy(Node* node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }
public:
    ~StringIntMap() {
        destroy(root);
    }
    void insert(const string& key, int value) {
        root = insert(root, key, value);
    }
    bool get(const string& key, int& out) const {
        return get(root, key, out);
    }
    void display() const {
        inorder(root);
    }
};
```

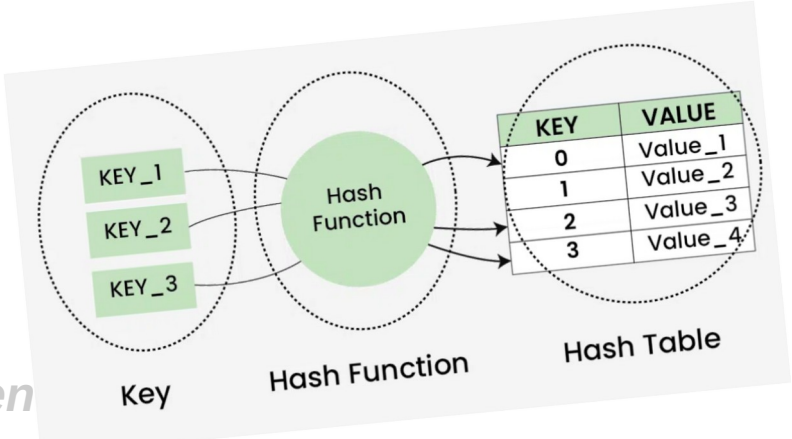
Main

```
int main() {
    StringIntMap students;

    students.insert("Aline", 19);
    students.insert("Kwizera", 22);
    students.insert("Mukamana", 20);
    students.insert("Niyonkuru", 21);
    students.insert("Iradukunda", 23);
    students.insert("Uwase", 18);
    students.insert("Hirwa", 24);
    students.insert("Ishimwe", 20);

    cout << " Students and their ages (in order):" << endl;
    students.display();

    cout << endl;

    int age;
    string query = "Mukamana";
    if (students.get(query, age)) {
        cout << query << " afite imyaka: " << age << endl;
    } else {
        cout << query << " ntiwabonywe." << endl;
    }

    return 0;
}
```

```
 Students and their ages (in order):
Aline: 19
Hirwa: 24
Iradukunda: 23
Ishimwe: 20
Kwizera: 22
Mukamana: 20
Niyonkuru: 21
Uwase: 18

Mukamana afite imyaka: 20
```

Hash Tables

A Hash table is defined as a data structure used to insert, look up, and remove key-value pairs quickly. It operates on the **hashing concept**, where each key is translated by a hash function into a distinct index in an array. The index functions as a storage location for the matching value. In simple words, it maps the keys with the value



Terms

Hash Function – Maps a key to an index in the array.

Array (bucket array) – Stores entries at indices determined by the hash function.

1. **Collision Handling** – Multiple keys may map to the same index; strategies are needed to handle this:
    - **Chaining**: Use a linked list (or list) at each array index.
    - **Open Addressing**: Find another open slot using probing (linear, quadratic, etc.).

Hash Function

A Function that translates keys to array indices is known as a hash function. The keys should be evenly distributed across the array via a decent hash function to reduce collisions and ensure quick lookup speeds.

TYpes of Hashing Functions

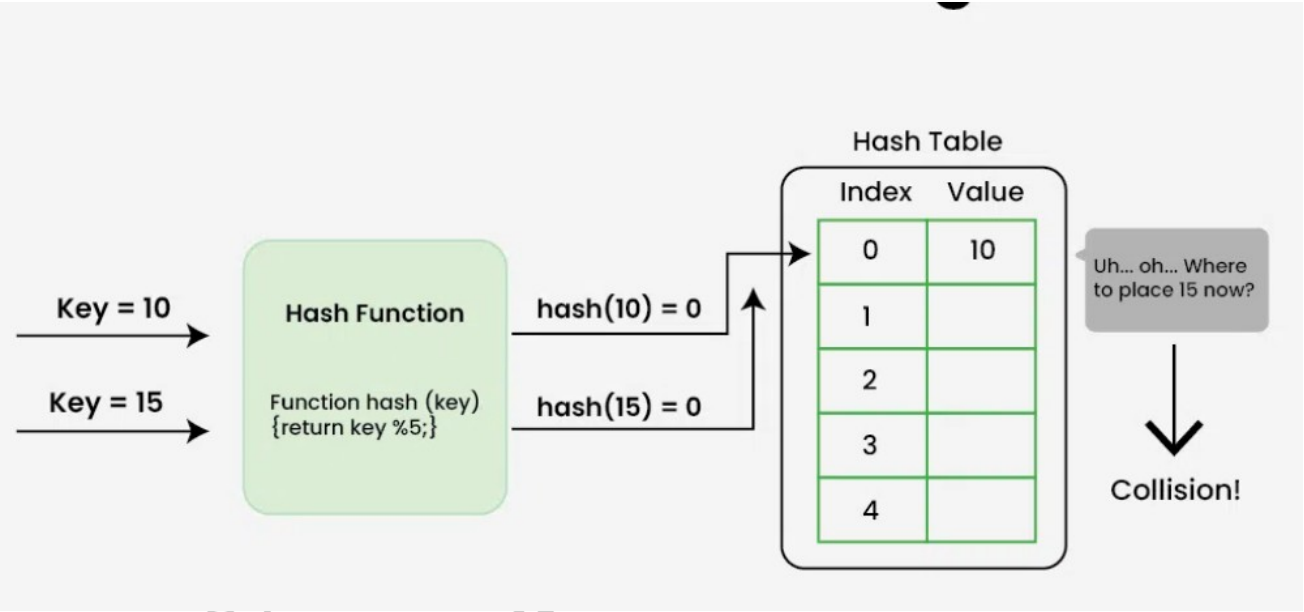
Integer universe assumption: Keys are assumed to be integers within a fixed range.

Hashing by division: Uses the remainder of key ÷ array size as the index.

Hashing by multiplication: Multiplies the key by a constant ($0 < c < 1$), takes the fractional part, and multiplies it by the array size to get the index.

Collisions Handling

Collisions happen when two or more keys point to the same array index. Chaining, open addressing, and double hashing are a few techniques for resolving collisions.



Collision handling

Separate Chaining: In separate chaining, a linked list of objects that hash to each slot in the hash table is present. Two keys are included in the linked list if they hash to the same slot. This method is rather simple to use and can manage several collisions.

Collision handling

Open Addressing:: collisions are handled by looking for the following empty space in the table. If the first slot is already taken, the hash function is applied to the subsequent slots until one is left empty. There are various ways to use this approach, including double hashing, linear probing, and quadratic probing.

Hash Table complexity

Hash allows fast insertion, deletion, and lookup operations (typically **O(1)** time complexity on average).

Example #1

```
#include <iostream>
#include <list>
#include <vector>
#include <string>
Using namespace std;
class HashTable {
Private:
// Number of buckets
    static const int HASH_GROUPS = 10;
 vector<list<pair<int,string>>> table;

public:
    HashTable() {
        table.resize(HASH_GROUPS);
    }

    int hashFunction(int key) {
        return key % HASH_GROUPS;
    }
```

```
    void insert(int key, const string &value)
{
        int hashValue = hashFunction(key);
        auto &cell = table[hashValue];
        for (auto &pair : cell) {
            if (pair.first == key) {
                pair.second = value;
                return;
            }
        }
        cell.emplace_back(key, value);
    }
void remove(int key) {
    int hashValue = hashFunction(key);
    auto &cell = table[hashValue];
    for (auto it = cell.begin(); it != cell.end(); ++it) {
        if (it->first == key) {
            cell.erase(it);
            return;
        }
    }
}
```

Example#1 next

```
string search(int key) {
        int hashValue = hashFunction(key);
        auto &cell = table[hashValue];
        for (const auto &pair : cell) {
            if (pair.first == key) {
                return pair.second;
            }
        }
        return "Key not found";
    }

    void display() {
        for (int i = 0; i < HASH_GROUPS; ++i) {
            cout << "Bucket " << i << ": ";
            for (const auto &pair : table[i]) {
                cout << "[" << pair.first << " : " << pair.second << "] ";
            }
            cout << "\n";
        }
    }
};
```

Example #1 Main Program

```
int main() {
    HashTable ht;

    ht.insert(905, "John");
    ht.insert(201, "Jane");
    ht.insert(332, "Tom");
    ht.insert(32, "Ange");
    ht.insert(124, "Mike");
    ht.insert(154, "Michael");
    ht.insert(127, "Chael");
    ht.insert(107, "Alice");

    ht.display();

    cout << "Searching for key 201: " << ht.search(201) << "\n";

    ht.remove(201);
    cout << "After removing key 201:\n";
    ht.display();

    return 0;
}
```

Example 2#

```
#include <iostream>
#include <list>
#include <string>

using namespace std;

struct Student {
    int id;
    string name;
    int age;
    string school;
};
class SchoolHashTable {
private:
    static const int SIZE = 10;
    list<Student> table[SIZE];

    int hash(int id) {
        return id % SIZE;
    }
```

```
public:
    void insert(Student student) {
        int index = hash(student.id);
        table[index].push_back(student);
    }

    void search(int id) {
        int index = hash(id);
        for (Student s : table[index]) {
            if (s.id == id) {
                cout << "Found student: "
                     << s.id << ", " << s.name << ", "
                     << s.age << ", " << s.school << "\n";
                return;
            }
        }
        cout << "Student with ID " << id << " not found.\n";
    }
  void display() {
        for (int i = 0; i < SIZE; ++i) {
            cout << "Bucket " << i << ": ";
            for (Student s : table[i]) {
                cout << "(" << s.id << ", " << s.name << ", "
                     << s.age << ", " << s.school << ") ";
            }
            cout << "\n";
        }  }
};
```

Example 2#next

```
int main() {
    SchoolHashTable ht;

    ht.insert({101, "Divine", 20, "RCA"});
    ht.insert({102, "Mugisha", 21, "RCA"});
    ht.insert({132, "Mugabo", 21, "RCA"}); // goes to same bucket as 102
    ht.insert({111, "Charlie", 19, "GS KAZUBA"});  // goes to same bucket as
101
 ht.insert({139, "Chance", 23, "RCA"});
  ht.insert({135, "Mike", 18, "RCA"});
    ht.display();

    ht.search(102);
    ht.search(999); // not found

    return 0;
}
```

Output

```
damascene10@damascene10-Latitude-3400:~$ g++ -o hashTable hash.cpp
damascene10@damascene10-Latitude-3400:~$ ./hashTable
Bucket 0:
Bucket 1: (101, Divine, 20, RCA) (111, Charlie, 19, GS KAZUBA)
Bucket 2: (102, Mugisha, 21, RCA) (132, Mugabo, 21, RCA)
Bucket 3:
Bucket 4:
Bucket 5: (135, Mike, 18, RCA)
Bucket 6:
Bucket 7:
Bucket 8:
Bucket 9: (139, Chance, 23, RCA)
Found student: 102, Mugisha, 21, RCA
Student with ID 999 not found.
```

Custom Map

We'll create a simple MyMap<Key, Value> class that:

- Stores key-value pairs
- Uses a **hash table with chaining** (linked lists) to handle collisions
- Supports operations: insert(key, value), get(key), and remove(key)

```
#include <iostream>
#include <list>
#include <vector>
#include <utility>  // for std::pair

using namespace std;

template <typename K, typename V>
class MyMap {
private:
    static const int TABLE_SIZE = 10;

    // Each bucket is a list of (key, value) pairs
    vector<list<pair<K, V>>> table;

    int hashFunction(const K& key) const {
        return hash<K>()(key) % TABLE_SIZE;  // Use
std::hash for key hashing
    }
```

Custom Map#next

```
public:
    MyMap() {
        table.resize(TABLE_SIZE);
    }
    void insert(const K& key, const V& value)
{
        int index = hashFunction(key);
        auto& cell = table[index];
        for (auto& pair : cell) {
            if (pair.first == key) {
                pair.second = value;  // update
value if key exists
                return;
            }
        }
        cell.emplace_back(key, value);  //
insert new key-value pair
    }
```

```
bool get(const K& key, V& valueOut) const {
        int index = hashFunction(key);
        const auto& cell = table[index];

        for (const auto& pair : cell) {
            if (pair.first == key) {
                valueOut = pair.second;
                return true;
            }
        }
        return false;  // key not found
    }

    void remove(const K& key) {
        int index = hashFunction(key);
        auto& cell = table[index];
        for (auto it = cell.begin(); it != cell.end(); ++it) {
            if (it->first == key) {
                cell.erase(it);
                return;
            }
        }

        cout << "Key not found: " << key << "\n";
    }

    void display() const {
        for (int i = 0; i < TABLE_SIZE; ++i) {
            cout << "Bucket " << i << ": ";
            for (const auto& pair : table[i]) {
                cout << "(" << pair.first << ": " << pair.second << ") ";
            }
            cout << "\n";
        }
    }
};
```

Main

```
int main() {
    MyMap<string, int> ages;

    ages.insert("Kevine", 25);
    ages.insert("Davine", 30);
    ages.insert("Kevin", 28);
    ages.insert("Joy", 26);  // Updates existing entry

    ages.display();

    int val;
    if (ages.get("Joy", val)) {
        cout << "Joys " << val << " years old.\n";
    } else {
        cout << "Joy not found.\n";
    }

    ages.remove("Davine");
    ages.display();

    return 0;
}
```